

Hands-on Activity 4.1	
Functions, Tuples, Dictionaries, Exceptions, and Data Processing	
Course Code: CPE009A	Program: Computer Engineering
Course Title: Object Oriented Programming 1	Date Performed:1/14/26
Section:CPE12S1	Date Submitted:1/20/26
Name(s):Niel Vincent B. Condino	Instructor: Engr. Jimlord M. Quejado
6. Output	

```

def is_year_leap(year):
    if (year % 4) > 0: #Common
        return False
    elif (year % 100) > 0: #Leap
        return True
    elif (year % 400) > 0: #Common
        return False
    else: #Leap
        return True

#Test checker
test_data = [1900, 2000, 2016, 1987]
test_results = [False, True, True, False]

for i in range(len(test_data)):
    yr = test_data[i]

    print(yr, "->", end="")

    result = is_year_leap(yr)

    if result == test_results[i]:
        print("OK")
    else:
        print("Failed")

... 1900 ->OK
    2000 ->OK
    2016 ->OK
    1987 ->OK

```

Figure 1: Code Snippet of Task 1.1 and its output

The figure presents the use of the function `is_year_leap` to detect if a given year is a leap year. It works by following the leap year logic of: if the year number isn't divisible by four, it's a common year; otherwise, if the year number isn't divisible by 100, it's a leap year; otherwise, if the year number isn't divisible by 400, it's a common year; otherwise, it's a leap year. After it checks the given argument using the logic stated before, it returns a bool value, True if it is a leap year and False if not. To check if this function is behaving as expected, a tester code is used. It checks if the returned value of the function is the same as the expected value. It prints "OK" if the value is expected and "Failed" if it doesn't. As shown by the output, it printed all 4 cases with "OK" meaning it passed the test and returned the expected value.

```

def is_year_leap(year) -> bool:
    if (year % 4) > 0: #Common
        return False
    elif (year % 100) > 0: #Leap
        return True
    elif (year % 400) > 0: #Common
        return False
    else: #Leap
        return True

def days_in_month(year, month) -> int:
    day_list = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
    if month > len(day_list): return #return none if month does not makes sense
    if (month == 2) and (is_year_leap(year)):
        day_list[1] = 29 #change feb to 29 cuz leap year
    return day_list[month - 1] #return the right index

#For testing purpose
test_years = [1900, 2000, 2016, 1987]
test_months = [2, 2, 1, 11]
test_results = [28, 29, 31, 30]

for i in range(len(test_years)):
    yr = test_years[i]
    mo = test_months[i]
    print(yr, mo, "->", end="")
    result = days_in_month(yr, mo)
    if result == test_results[i]:
        print("OK")
    else:
        print("Failed")

```

Figure 2.1: Code Snippet of Task 1.2

```

... 1900 2 ->OK
    2000 2 ->OK
    2016 1 ->OK
    1987 11 ->OK

```

Figure 2.2: Output of Code from Figure 2.1

The figure above shows the code for the function of figuring out days given a month. This code utilizes the function from the last task for checking if a year is a leap year. To see how the first function works, refer to the discussion of the first figure. The second function, checks the number of days given a month. First, it has a list that contains the value of the number of days for each month. For example, the first value corresponds to the number of days for the first month and so on. After this, it checks if the month given is not in range of the list, if it is, then it returns None and exits the function early. Next, it checks if the month is 2 and it is a leap year. Due to February having 29 days when it's leap year, it modifies the value of the second index to match it. Lastly, it returns the value in the index that has the same value as the month lessened by one. This is to achieve the outcome expected. For example, if the month in the argument is 3, then it returns the value on the index 2 which is the 3rd value. Below this function is a test case code. As shown by the output, it passed all the test cases. Therefore, this function is working as expected.

```

def is_year_leap(year) -> bool:
    if (year % 4) > 0: #Common
        return False
    elif (year % 100) > 0: #Leap
        return True
    elif (year % 400) > 0: #Common
        return False
    else: #Leap
        return True

def days_in_month(year, month) -> int:
    day_list = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
    if month > len(day_list): return #return none if month does not makes sense
    if (month == 2) and (is_year_leap(year)):
        day_list[1] = 29
    return day_list[month - 1]

def day_of_year(year, month, day) -> int:
    c_days = 0 # cumulative days
    for month_count in range(month):
        dy_in_mo = days_in_month(year, month_count+1)
        if month_count == month - 1: #reached the month given in the argument
            if day > dy_in_mo: #Given day is greater than num of days month in that month
                return #The argument does not make sense
            else:
                c_days += day
        else:
            c_days += dy_in_mo
    return c_days

#For testing purpose
test_years = [1900, 2000, 2016, 1987]
test_months = [2, 2, 1, 11]
test_days = [20, 29, 20, 30]
test_results = [51, 60, 20, 334]

for i in range(len(test_years)):
    yr = test_years[i]
    mo = test_months[i]
    dy = test_days[i]
    result = day_of_year(yr, mo, dy)
    print(yr, mo, dy, "->", f"({result})", end="")
    if result == test_results[i]:
        print("OK")
    else:
        print("Failed")

```

Figure 3.1: Code snippet for Task 1.3

```

• 1900 2 20 -> (51)OK
  2000 2 29 -> (60)OK
  2016 1 20 -> (20)OK
  1987 11 30 -> (334)OK

```

Figure 3.2: Output of the code from Figure 3.1

Figure 3.1 shows the different functions from the previous figure and the new function which takes three arguments (a year, a month, and a day of the month) and returns the corresponding day of the year, or returns None if any of the arguments is invalid. The function that is presented on the figure runs on this logic: first it creates a cumulative day variable. Then it loops for each month, It then calculates the value of the days in the given month using the function in figure 2 and stores it inside a variable `dy_in_mo`. It then checks if the current month is the last month from the given argument. If it is, then it checks if the days argument is greater than the days in the month, if it is, then it returns None and exits the function. If not, it just adds the value of this argument to the cumulative day variable. If it is not the last month, then it adds the value calculated by the second function for the max number of days for that month. After this loop is finished, it returns the value of the cumulative days. The bottom code is used for testing the function to see if it does return the expected value. As seen in figure 3.2, it has passed the test and returned the expected values. This means the function is working as expected.

```
def is_prime(num):
    for i in range(2,num//2):
        if num % i == 0:#check if divisible/has factors
            return False
    return True

for i in range(1,20):
    if is_prime(i+1):
        print(i+1,end=" ")

2 3 4 5 7 11 13 17 19
```

Figure 4: Code snippet for a Task 2.1, prime checker and its output

Figure 4 presents the code for a function that is used to check if a given number is a prime number. In the function `is_prime`, it runs through a loop. This loop iterates from 2 to the half of the given number. It only just checks the first half of the factors as looping through the other half just seems redundant. This way, it cuts the time it needs to check. If the number is perfectly divisible to any number given by the loop, it returns false. If not, it is assumed to have no factors and therefore returns true. This function is tested by printing the prime numbers from 1 to 20. As shown by its output, it has passed the test and just prints all the primes from the range of the numbers given.

```
#1 American mile = 1609.344 metres;
#1 American gallon = 3.785411784 litres.

def liters_100km_to_miles_gallon(liters):
    #convert to mil/gal
    gal = liters/3.785411784
    mil = 100000/1609.344
    return mil/gal

def miles_gallon_to_liters_100km(miles):
    km = (miles * 1609.344) / 100000

    lit = 3.785411784
    return lit/km

print(liters_100km_to_miles_gallon(3.9))
print(liters_100km_to_miles_gallon(7.5))
print(liters_100km_to_miles_gallon(10.))
print(miles_gallon_to_liters_100km(60.3))
print(miles_gallon_to_liters_100km(31.4))
print(miles_gallon_to_liters_100km(23.5))

... 60.31143162393162
    31.361944444444444
    23.521458333333333
    3.900739358761747
    7.490910297239916
    10.009131205673759
```

Figure 5:Code snippet and output of Task 2.2 (l/100km-mpg and vice versa converter)

This figure presents a code that has different functions used to convert liters per 100 km to miles per gallon and vice versa. These functions are separate from one another. Using the conversion factors commented in the top most section of the code, The first function converts l/100km. The function does this by converting the given liters argument to gallons, then, it converts 100km to miles. Lastly this function returns the value of miles divided by gallons. The next function converts miles per gallon to liters/100km. First, this function uses the miles argument and converts it to km then divides it by 100km. Next it uses the constant conversion factor of gallon to liters. Lastly this returns the value of liters per 100 km. The print functions below are for testing. As shown by the figure, the output of the code matches the expected output from the task for the functions. Therefore the function is working as expected

7. Supplementary Activity

```
def compute_average(grades):  
    c_sum = 0  
    grade_sz = len(grades)  
  
    for i in range(grade_sz): #summation of grades  
        c_sum += grades[i]  
  
    mean = c_sum / grade_sz #average  
    rep_mean = round(mean,2) #round  
  
    return rep_mean  
  
grade = (84.5,87.3,93,90,95)  
print(compute_average(grade))  
  
... 89.96
```

Figure 6: Code snippet for average computer function

This figure shows a function that, as the name suggests, computes the average of a given tuple of grades. This function has a parameter named “grades” that accepts a tuple. It first gets the summation of all the grades then divides it by the number of grades. The result will be then rounded to the second digit. Lastly, this function returns the rounded average of the grades. Below this a test case of the function. Given a value of grades inside a tuple, the function is called and then printed in the output box. As shown, the output is rounded to the second decimal value.

```
def get_price(products,item_name):  
  
    price = products.get(item_name)  
  
    if price is None:# if not found  
        return "Item does not exist"  
    else:  
        return ("Price: " + str(price))  
  
products_sold = {  
    "gun": 500,  
    "cigar": 2,  
    "hat":25,  
    "boots":37,  
}  
  
print(get_price(products_sold,input("Input item name:")))
```

Figure 7.1: Code snippet for price finder function

Figure 7.1 shows a function that accepts products dictionary and item name string as the parameter and returns the price of the given item. This function first gets the price of the product by using the .get() built-in function for dictionaries in python and stores it inside a variable named price. Then it checks if the value of the price variable is None. If it is then it returns a message saying the item does not exist. If not then it returns the price of the item found in the dictionary. This function is then tested using a test dictionary named “products_sold”. The function is called with the arguments products_sold dictionary and the item_name’s value is inputted by the user.

Input item name:gun Price: 500	Input item name:cigar Price: 2
Input item name:hat Price: 25	Input item name:horse Item does not exist

Figure 7.2: Output for price finder function

This figure shows the output for the code in Figure 7.1. As shown, It is tested using different inputs. The first three, gun, cigar, and hat, all exist in the dictionary. This caused the function to return their price from the dictionary. In the last image, the user inputted a horse. Because the string "horse" does not exist in the dictionary, it just returned the "Item does not exist" string.

```
def analyze_sales(sales_data):
    t_sales = 0 #total
    h_sale = 0 # highest sale

    for sale in sales_data.values():
        t_sales += sale #compile all sales
        if h_sale < sale:
            h_sale = sale #get the highest sale

    a_sale = t_sales / len(sales_data) #average sale

    sale_data = {
        "Total Sale":t_sales,
        "Average Sale":a_sale,
        "Highest Sale":h_sale,
    }

    return sale_data #return dictionary

daily_sale = {
    "Monday":200,
    "Tuesday":300,
    "Wednesday":260,
    "Thursday":700,
    "Friday":500,
    "Saturday":1000,
    "Sunday":600
}

sale_det = analyze_sales(daily_sale)

print("Total Sale:",sale_det["Total Sale"])
print("Average Sale:",sale_det["Average Sale"])
print("Highest Sale:",sale_det["Highest Sale"])

... Total Sale: 3560
Average Sale: 508.57142857142856
Highest Sale: 1000
```

Figure 8: Code snippet and output for a function for analyzing sales

Figure 8 shows the code for the function named `analyze_sales`. This function takes a dictionary for `sales_data` as an argument. To get the total sales and highest sale, it loops through each value of the dictionary, for the total sale it adds them all up and for the highest sale, it checks if the current sale value is higher than the last sale value. If it is then it changes the highest sale variable's value. The average is computed using the total sales divided by the number of values inside the dictionary. Then it creates another dictionary named `sale_data`. Inside this dictionary the values of total sales, highest sale, and average sale is stored. Lastly, the value of this dictionary is returned. After this the function is tested using a `daily_sale` dictionary. The function is called using the `daily_sale` as the argument and the value is stored inside the `sale_det` variable. The series of print functions are used to output the values inside this dictionary.

8. Conclusion

In this activity I was able to use the different types of data storage in python that are not a list. These are tuples, and dictionaries. I find these very useful for specific use cases. Tuples can be used if I do not want any modification inside a defined array. The dictionaries use a very interesting gimmick in which the index is defined by the programmer instead of being the traditional numbers 0 to number of items minus 1. This can be useful to name an index of a data storage so that it is clearer what it stores. This improves the code's readability. The functions are also discussed. The only difference I see they have against their C++ counterpart is that: it is less stricter with data type and uses indention instead of enclosing the block of code inside a curly bracket to define its scope. The different tasks and supplementary tasks also enhanced my understanding of these programming concepts. All of the tasks aimed to put all the code inside a function in order to familiarize the students with how functions work. These tasks also utilized the concept of dictionary and tuples in them. I used these concepts to achieve the desired output of each task. Adding to this, even though the concept of try except function is discussed, I did not see the need to use it because most of the functions are not waiting for user inputs which most of the time is the main cause of these exceptions. Overall, I think I understood the concepts taught for these activities and I know these concepts are also the basics of python which will be essential with the journey down the road.

9. Assessment Rubric